

Program Collection

Collection of Programming Examples

WAP to print Hello World!! on the screen using C++.

```
#include<iostream>
int main()
{
    std::cout << "Hello World!!" << std::endl;
    return(0);
}
```

or

```
#include<iostream>
using namespace std;
int main()
{
    cout << "Hello World!!" << endl;
    return(0);
}
```

WAP to find LCM two given number using recursion

```
#include <iostream>
using namespace std;
int lcm(int m, int n, int a)
{
    if((a%m == 0) && (a%n == 0))
        return(a);
    else
        return(lcm(m,n, a+n));
}
int main()
{
    int m, n;
    cout << "Enter the 1st Num :";
    cin >> m;
    cout << "Enter the 2nd Num :";
    cin >> n;
    cout << "LCM of " << m << " & " << n << " is : " << lcm(m,n,n) <<
endl;
    return(0);
}
```

WAP to get 5 student details using structure and print on the screen

```
//WAP to get 5 student details using structure and print on the screen
```

```
#include<iostream>
#include<iomanip>
using namespace std;
struct student
{
    char name[30];
    int roll;
    float cgpa;
};
int main()
{
    struct student s[5];
    for(int i=0;i<5;i++)
    {
        cout << "Name of " << i << "th student : ";
        if(i!=0)
            cin.ignore();
        cin.getline(s[i].name, 30);
        cout << "Roll of " << i << "th student : ";
        cin >> s[i].roll;
        cout << "cgpa of " << i << "th student : ";
        cin >> s[i].cgpa;
    }

    // Print header with column titles
    cout << left << setw(30) << "Name"      // Left align, width 30
         << setw(10) << "Roll No"         // Left align, width 10
         << setw(10) << "cgpa"            // Left align, width 10
         << endl;

    // Print a separator line
    cout << setfill('-') << setw(50) << "-" << endl;
    cout << setfill(' '); // Reset fill to space

    // Print data in columns
    for (int i = 0; i < 5; i++) {
        cout << left << setw(30) << s[i].name
             << setw(10) << s[i].roll
             << fixed << setprecision(2) << setw(10) << s[i].cgpa
             << endl;
    }
    return(0);
}
```

Write a code to create a student class and provide appropriate functions to set, display and modify cgpa with a public driver function.

```
#include<iostream>
using namespace std;

class student{
    int id;
    char name[30];
    float cgpa;
    void getdata()
    {
        cout << "Eneter id : ";
        cin >> id;
        cout << "Enter name : ";
        cin.ignore();
        cin.getline(name, 30);
        cout << "Enter the CGPA : ";
        cin >> cgpa;
    }
    void printdata()
    {
        cout << "Name : " << name << endl;
        cout << "ID : " << id << endl;
        cout << "CGPA : " << cgpa << endl;
    }
    void changecgpa()
    {
        cout << "Enter new CGPA : ";
        cin >> cgpa;
    }
public:
    void driver()
    {
        int i;
        cout << "Enter 1 : for entring information " << endl;
        cout << "Enter 2 : for showing information " << endl;
        cout << "Enter 3 : for changing cgpa value " << endl;
        cout << "enter your choice :";
        cin >> i;
        switch(i)
        {
            case 1: getdata(); break;
            case 2: printdata(); break;
            case 3: changecgpa(); break;
            default: cout << "Wrong Choice ...." << endl;
        }
    }
}
```

```

    }
};

int main()
{
    student s1;
    for(int i=0;i<4;i++)
        s1.driver();
    return(0);
}

```

Write a code to use the above student class to store and display details of `n` numbers of students.

```

#include<iostream>
using namespace std;

class student{
    int id;
    char name[30];
    float cgpa;
    void getdata()
    {
        cout << "Eneter id : ";
        cin >> id;
        cout << "Enter name : ";
        cin.ignore();
        cin.getline(name, 30);
        cout << "Enter the CGPA : ";
        cin >> cgpa;
    }
    void printdata()
    {
        cout << "Name : " << name << endl;
        cout << "ID : " << id << endl;
        cout << "CGPA : " << cgpa << endl;
    }
    void changecgpa()
    {
        cout << "Enter new CGPA : ";
        cin >> cgpa;
    }
public:
    void driver(int i)
    {
        switch(i)
        {
            case 1: getdata(); break;
            case 2: printdata(); break;

```

```

        case 3: changeGpa(); break;
        default: cout << "Wrong Choice ...." << endl;
    }
}
};
int main()
{
    int n;
    cout >> "Enter the number of students : ";
    cin << n;
    student s[n];
    // Store the details
    for(int i=0; i<n; i++)
        s[i].driver(1);
    // Show the details
    for(int i=0; i<n; i++)
        s[i].driver(2);
    return(0);
}

```

Write a program in C++ that simulates an online shopping cart system. Create a structure `item` having data members as `ItemName`, `ItemCost`, `ItemCount`. The `Cart` class needs item count from the user and uses item structure to store information. Use appropriate functions to set, print and calculate the total cost of the items.

```

#include<iostream>
using namespace std;

struct item{
    char ItemName[20];
    float ItemCost;
    int ItemCount;
};

class cart
{
private:
    int n;
    item *itm;
    float TotalCost;
public:
    void getData()
    {
        cout << "Enter the number of item :";
    }
}

```

```

        cin >> n;
        itm = new item[n];
    }
    void getItemDetails()
    {
        for(int i=0;i<n;i++)
        {
            cout << "Item Name : ";
            cin.ignore();
            cin.getline(itm[i].ItemName,20);
            cout << "Item Cost : ";
            cin >> itm[i].ItemCost;
            cout << "Item count : ";
            cin >> itm[i].ItemCount;
        }
    }
    void calculatePrice()
    {
        TotalCost = 0;
        for(int i=0;i<n;i++)
        {
            TotalCost = TotalCost + (itm[i].ItemCost * itm[i].ItemCount);
        }
        cout << "Total cost : " << TotalCost << endl;
    }
};
int main()
{
    cart c1;
    c1.getdata();
    c1.getItemDetails();
    c1.calculatePrice();
    return(0);
}

```

WAP for banking applications where each time a new account is created, the account holder's details and initial balance must be initialized using a constructor and when an account object goes out of scope, it should display a message saying the account is closed using a destructor.

```

#include <iostream>
#include <cstring>
using namespace std;

class BankAccount {
private:

```

```

char accountHolder[30];
int accountNumber;
double balance;

public:
    // Defining default constructor
    BankAccount()
    {
        cout << "Account creation process started ... " << endl;
    }
    // Constructor to initialize account details
    BankAccount(char *name, int accNum, double initialBalance) {
        strcpy(accountHolder, name);
        accountNumber = accNum;
        balance = initialBalance;
        cout << "Account created successfully!" << endl;
        cout << "Account Holder: " << accountHolder << endl;
        cout << "Account Number: " << accountNumber << endl;
        cout << "Initial Balance: " << balance << endl;
    }

    // Destructor to display message when object goes out of scope
    ~BankAccount() {
        cout << "Account " << accountNumber
             << " belonging to " << accountHolder
             << " is closed." << endl;
    }

    //void get data from the user
    void get_data()
    {
        cout << "Account Holder: " << endl;
        cin.ignore();
        cin.getline(accountHolder, 30);
        cout << "Account Number: " << endl;
        cin >> accountNumber;
        cout << "Initial Balance: " << endl;
        cin >> balance;
    }

    // Function to deposit money
    void deposit(double amount) {
        balance += amount;
        cout << "Deposited: " << amount
             << " | New Balance: " << balance
             << endl;
    }

    // Function to withdraw money
    void withdraw(double amount) {

```

```

        if (amount > balance) {
            cout << "Insufficient balance!" << endl;
        } else {
            balance -= amount;
            cout << "Withdrawn: " << amount
                << " | New Balance: " << balance
                << endl;
        }
    }

    // Function to display account details
    void display() {
        cout << "Account Holder: " << accountHolder << endl;
        cout << "Account Number: " << accountNumber << endl;
        cout << "Balance: " << balance << endl;
    }
};

int main() {
    // Creating an account
    BankAccount acc("Dibyasundar Das", 101, (double)(5000.0));
    acc.deposit(2000);
    acc.withdraw(1500);
    acc.display();
    // Account goes out of scope here, triggering destructor
    return 0;
}

```

For a library, you want to keep track of books being borrowed and returned. When a book is borrowed, its title, author, and borrower name are recorded. When the book is returned (object destroyed), display a message with the borrower name and book title.

```

#include <iostream>
#include <cstring>
using namespace std;

class LibraryBook {
private:
    char title[100];
    char author[100];
    char borrower[100];

public:
    // Constructor to initialize book details
    LibraryBook(const char* bookTitle, const char* bookAuthor, const

```

```

char* borrowerName) {
    strcpy(title, bookTitle);
    strcpy(author, bookAuthor);
    strcpy(borrower, borrowerName);

    cout << "Book Borrowed: " << title
         << " by " << author
         << " | Borrower: " << borrower
         << endl;
}

// Destructor to display message when book is returned
~LibraryBook() {
    cout << "Book Returned: " << title
         << " | Borrower: " << borrower
         << endl;
}
};

int main() {
    int choice;
    LibraryBook *book1;
    while(1)
    {
        cout << "Press 1 : Issue a book "<<endl;
        cout << "Press 2 : Return a book "<<endl;
        cin >> choice;
        // Borrowing a book
        if (choice ==1)
        {
            book1 = new LibraryBook("Wings of Fire",
                                     " Arun Tiwari", "Dibyasundar Das");
        }
        else
        {
            delete book1;
        }
    }

    return 0;
}

```

WAP to Display a window of size 800 × 600

```

#include<SFML/Graphics.hpp>
using namespace sf;

inr main()

```

```

{
    VideoMode video(800,600);
    RenderWindow window(video, "Window!");

    while(window.isOpen())
    {
        window.clear();
        window.display();
    }
}

```

Write a code snippet to close the window when close button is pressed.

```

while(window.isOpen())
{
    Event ev;
    while(window.pollEvent(ev))
    {
        if(ev.type == Event::closed)
        {
            window.close();
        }
    }
}

```

Write a code snippet to close the window when escape key is pressed.

```

while(window.isOpen())
{
    Event ev;
    while(window.pollEvent(ev))
    {
        if(ev.type == Event::KeyPressed && ev.key.code ==
Keyboard::Escape)
        {
            window.close();
        }
    }
}

```

OR

```
while(window.isOpen())  
{  
    if(Keyboard::isKeyPressed(Keyboard::Escape))  
    {  
        window.close();  
    }  
}
```